

Báo cáo Kỹ thuật

Hệ thống Cảnh báo Va chạm bằng Cảm biến Siêu âm JSN-SR04T

truyền dữ liệu trực tiếp bằng ESP-NOW

ACLAB

Ngày 10 tháng 8 năm 2026

Tóm tắt nội dung

Báo cáo này mô tả trạng thái hiện tại của hệ thống nhúng cảnh báo va chạm sử dụng cảm biến siêu âm chống nước JSN-SR04T, vi điều khiển ESP32-S3 và màn hình cảm ứng LVGL. Dữ liệu khoảng cách được truyền **trực tiếp giữa hai board bằng ESP-NOW** — liên kết Wi-Fi cục bộ không qua điểm truy cập, bộ định tuyến hay máy chủ đám mây — và ngưỡng cảnh báo được đánh giá cục bộ ngay trên thiết bị hiển thị. Nội dung tập trung vào phần cứng đang sử dụng, công cụ/framework và lý do lựa chọn, các API chính được gọi trong firmware, cũng như các chức năng đã hoàn thiện và hướng phát triển tiếp theo của hệ thống ở trạng thái hiện hành.

Mã nguồn: <https://github.com/YdtTran/supersonic-warning-system>

Mục lục

1	Giới thiệu và phạm vi hệ thống hiện tại	3
2	Kiến trúc hệ thống hiện hành	3
3	Phần cứng đang sử dụng	4
4	Công cụ và framework	6
5	Firmware Sensor Node	6
5.1	Kiến trúc tác vụ FreeRTOS	6
5.2	Bộ lọc khoảng cách (phân cụm + trung bình trượt mũ)	7
5.3	Truyền dữ liệu bằng ESP-NOW	7
5.4	Cảnh báo còi cục bộ	8
6	Firmware Waveshare Screen	8
6.1	Nhận dữ liệu ESP-NOW và đánh giá hazard cục bộ	8
6.2	Mô hình dữ liệu cảm biến	8
6.3	Giao diện LVGL 9.1	8
6.4	Lớp hỗ trợ phần cứng màn hình (BSP)	9

7	Kiến trúc CoreIoT trước đây (không còn sử dụng)	9
8	Chức năng đã hoàn thiện	10
9	Hướng phát triển tiếp theo	10

1. Giới thiệu và phạm vi hệ thống hiện tại

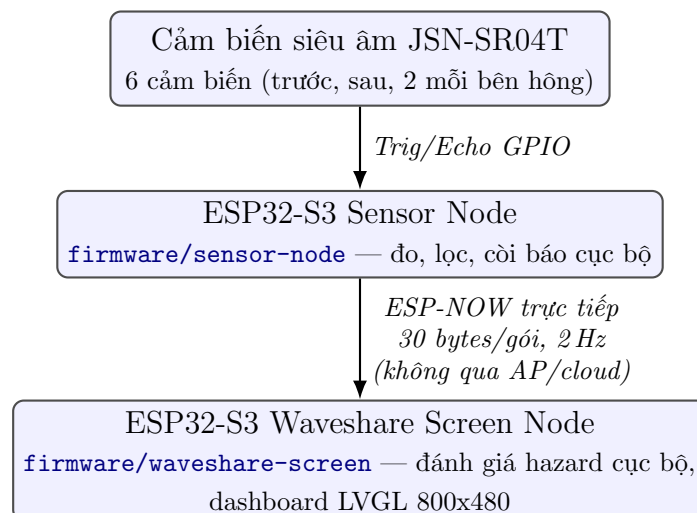
Hệ thống phát hiện vật cản/xe cộ ở khoảng cách gần bằng cảm biến siêu âm, truyền khoảng cách đã lọc trực tiếp sang node màn hình bằng **ESP-NOW**, nơi ngưỡng cảnh báo được đánh giá cục bộ và hiển thị theo thời gian thực; còi cảnh báo được điều khiển ngay trên node cảm biến.

Ở trạng thái hiện tại, hệ thống đã lắp đặt và vận hành trên phần cứng thật với **đầy đủ 6 cảm biến siêu âm** bao quanh xe (trước, sau, và hai cảm biến mỗi bên hông phủ nửa trước/nửa sau) theo đúng sơ đồ điểm mù (*no-zone*) của xe tải.

Phiên bản trước của hệ thống định tuyến dữ liệu qua nền tảng đám mây **CoreIoT (ThingsBoard)** bằng MQTT và xử lý ngưỡng bằng Rule-Chain. Kiến trúc này đã được thay thế bằng ESP-NOW nhằm loại bỏ phụ thuộc vào hạ tầng mạng và giảm độ trễ cảnh báo — một hệ thống an toàn gắn trên xe di chuyển không nên ngừng hoạt động khi mất sóng Wi-Fi hoặc Internet. Mã nguồn CoreIoT vẫn được giữ lại trong cây nguồn để có thể khôi phục, nhưng không nằm trên đường dữ liệu hiện hành (Mục 7).

2. Kiến trúc hệ thống hiện hành

Hình 1 mô tả luồng dữ liệu hiện hành của hệ thống: cảm biến siêu âm truyền tín hiệu Trig/Echo tới node cảm biến; node cảm biến lọc nhiễu rồi gửi khoảng cách đã lọc **trực tiếp** sang node màn hình bằng ESP-NOW; node màn hình tự đánh giá ngưỡng cảnh báo cục bộ và hiển thị. Toàn bộ đường dữ liệu nằm gọn giữa hai board, không có thành phần trung gian nào.

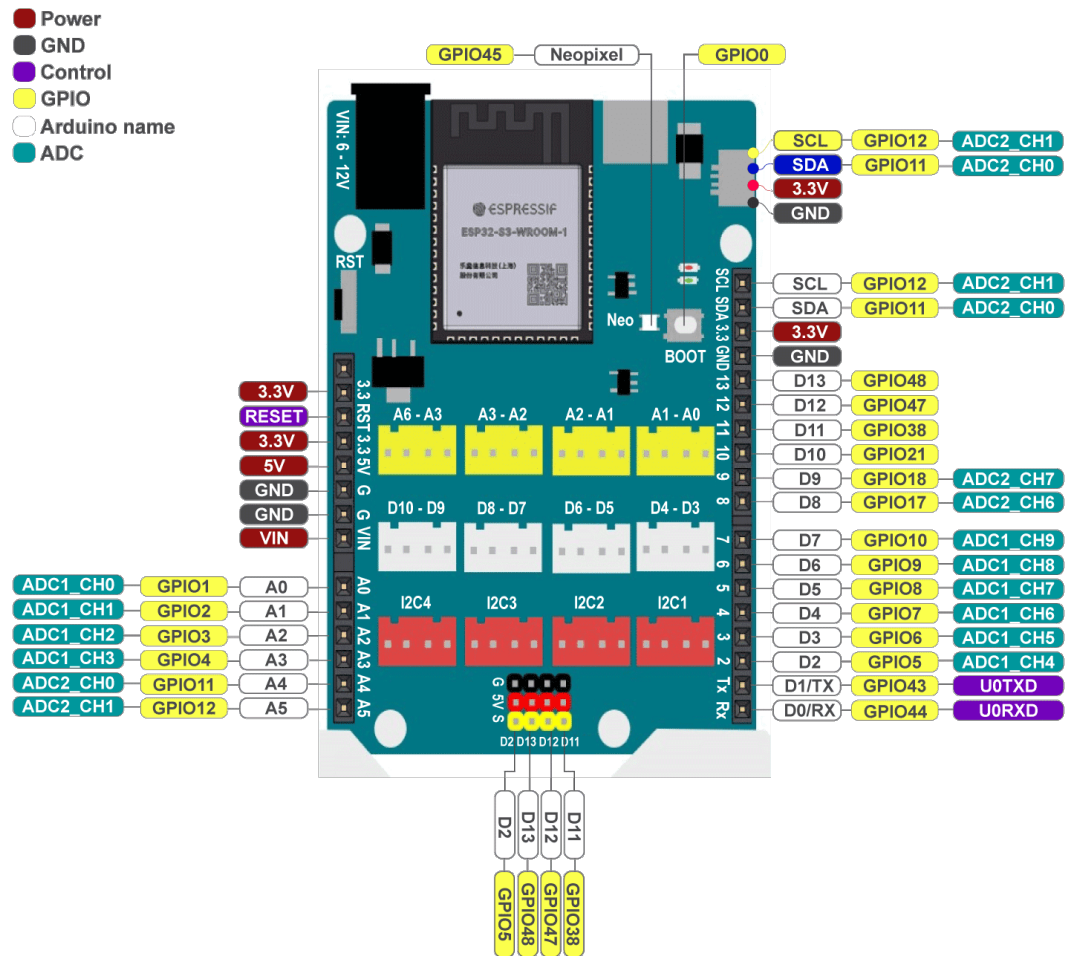


Hình 1: Luồng dữ liệu hiện hành của hệ thống.

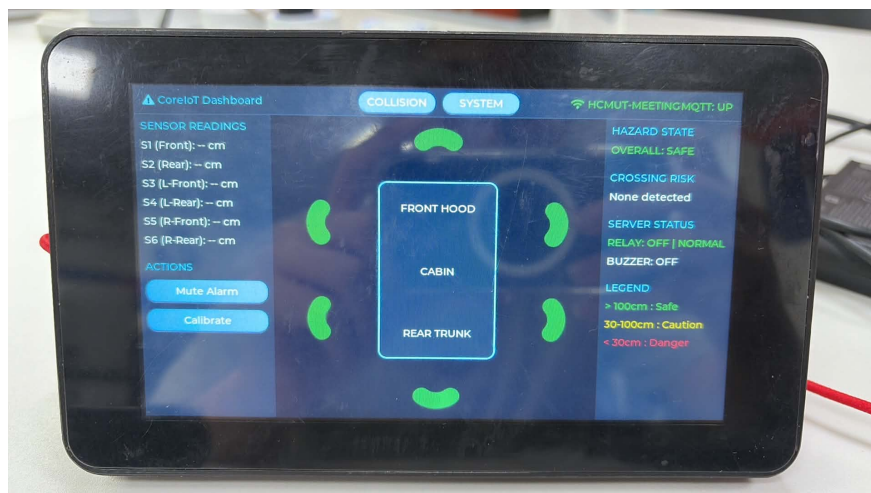
3. Phần cứng đang sử dụng

Thành phần	Mô tả
Vi điều khiển	Board Yolo:Uno , dựa trên ESP32-S3-WROOM-1 (Dual-Core 240MHz, Wi-Fi/BLE, PSRAM Octal).
Cảm biến khoảng cách	JSN-SR04T V3 — cảm biến siêu âm chống nước, đầu dò tách rời board mạch, chạy ở Mode 0 (vi điều khiển phát xung Trig, đo độ rộng xung Echo qua GPIO). Sử dụng 6 cảm biến .
Màn hình hiển thị	Waveshare ESP32-S3 Touch LCD 7" — panel RGB 800 × 480, cảm ứng dung kháng GT911 (I2C 400kHz), IO-expander CH422G (I2C, địa chỉ 0x24/0x38) điều khiển backlight, reset cảm ứng, CS thẻ SD và MUX CAN.
Cảnh báo cục bộ	Còi buzzer trên chân GPIO11 của Sensor Node, phản hồi tức thời không phụ thuộc đường truyền.
Ràng buộc chân GPIO	Chip ESP32-S3 bản <i>Embedded PSRAM 8MB</i> chiếm GPIO47/48 làm xung nhịp vì sai cho PSRAM; các chân này không dùng được cho tín hiệu Trig/Echo (Mục 9).

Bảng 1: Danh mục phần cứng chính đang sử dụng.



Hình 2: Sơ đồ pinout board Yolo:Uno (ESP32-S3).



Hình 3: Dashboard cảnh báo va chạm chạy trên màn hình Waveshare LCD 7"(phần cứng thật).

4. Công cụ và framework

Hạng mục	Lựa chọn	Lý do
Build system	PlatformIO (<code>pio run -e yolo_uno</code>)	Layout thống nhất cho cả hai firmware, một toolchain quản lý hai dự án khác framework.
sensor-node	<code>framework = arduino</code>	Tái sử dụng hệ sinh thái thư viện Arduino-ESP32 (<code>WiFi.h</code> , <code>esp_now.h</code>) — đủ cho tác vụ đo/lọc/gửi ESP-NOW.
waveshare-screen	<code>framework = espidf</code> thuần	Driver màn hình/cảm ứng hiện đại (<code>esp_lvgl_adapter</code> , <code>esp_lcd_touch_gt911</code> , LVGL 9.1, <code>esp_lcd_panel_rgb</code>) đòi hỏi ESP-IDF ≥ 5.5 .
Đồ hoạ UI	LVGL 9.1 + <code>esp_lvgl_adapter</code>	Thư viện GUI nhúng mã nguồn mở phổ biến cho ESP32, adapter chính thức của Espressif quản lý vòng lặp render và khoá luồng an toàn.
Liên kết hai board	ESP-NOW (giao thức lớp 2 trên nền Wi-Fi radio)	Không cần AP/router/broker: hệ thống hoạt động cả khi không có mạng, độ trễ mỗi gói dưới ≈ 10 ms, phù hợp cảnh báo va chạm thời gian thực.
Nền tảng IoT (<i>không còn dùng</i>)	CoreIoT (ThingsBoard)	MQTT-native, Rule-Chain kéo-thả xử lý ngưỡng không cần backend riêng. Đã thay bằng ESP-NOW kết hợp đánh giá hazard cục bộ.

Bảng 2: Công cụ, framework sử dụng và lý do lựa chọn.

5. Firmware Sensor Node

5.1. Kiến trúc tác vụ FreeRTOS

Bảng 3 liệt kê các tác vụ FreeRTOS đang chạy trên `sensor-node`.

Task	Stack	Priority	Core	Vai trò
SensorTask	4096	2	1	Trigger + đọc tuần tự từng cảm biến, đưa qua bộ lọc, ghi kết quả vào <code>SharedState</code> .
AppTask	2048	1	1	Tiêu thụ dữ liệu đã lọc song song, đọc lập chu kỳ đo.
NetworkTask	4096	1	0	Đóng gói <code>SharedState</code> và gửi ESP-NOW sang node màn hình mỗi 500 ms, tách khỏi core 1 để không ảnh hưởng timing đo.
BuzzerTask	2048	1	0	Đọc khoảng cách gần nhất, điều khiển còi vật lý GPIO11 theo ngưỡng WARNING/DANGER.

Bảng 3: Tác vụ FreeRTOS trên `sensor-node`.

5.2. Bộ lọc khoảng cách (phân cụm + trung bình trượt mũ)

Bộ lọc kết hợp phân cụm mẫu đo (loại outlier) và làm mượt bằng EMA (*Exponential Moving Average*) để giảm nhiễu vốn có của cảm biến siêu âm giá rẻ mà vẫn phản ứng nhanh với thay đổi khoảng cách thật. Bảng 4 tóm tắt các tham số chính.

Tham số	Ý nghĩa
Kích thước lịch sử	9 mẫu RAW gần nhất được lưu để phân cụm.
Cụm hợp lệ tối thiểu	5/9 mẫu phải thuộc cùng một cụm mới được chấp nhận.
Dung sai cụm	Cơ bản 8 cm, tăng theo tỉ lệ 8% so với khoảng cách đo (nhiều tỉ lệ thuận khoảng cách).
Hệ số làm mượt EMA	0.30.
Xác nhận bước nhảy	Một thay đổi lớn (>30 cm hoặc >25%) phải lặp lại 3 lần liên tiếp mới được chấp nhận, tránh nhiễu tức thời.
Reset bộ lọc	Sau 15 lần đọc lỗi liên tiếp.

Bảng 4: Tham số chính của bộ lọc khoảng cách.

5.3. Truyền dữ liệu bằng ESP-NOW

Node cảm biến gửi trực tiếp tới địa chỉ MAC của node màn hình bằng `esp_now_send()` (mã nguồn: [EspNowClient.cpp](#)). Các đặc điểm chính:

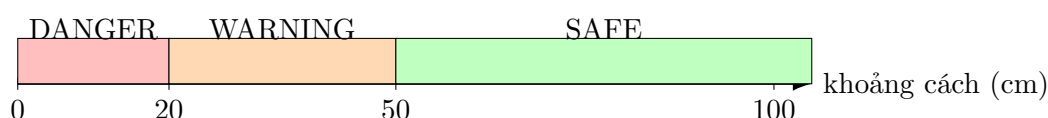
- `WiFi.mode(WIFI_STA)` chỉ để kích hoạt radio Wi-Fi; **không** gọi `WiFi.begin()` — không kết nối điểm truy cập nào, không broker, không phụ thuộc hạ tầng mạng.
- Kênh Wi-Fi cố định (kênh 1), bắt buộc trùng nhau ở cả hai board; địa chỉ MAC đích khai báo trong `EspNowConfig.h`.
- Payload là cấu trúc nhị phân đóng gói chặt (*packed*) gồm `float[6]` khoảng cách và `uint8_t[6]` cờ hợp lệ, tổng **30 byte** — chỉ chiếm 12% giới hạn 250 byte mỗi gói của ESP-NOW v1.0. Cờ hợp lệ bằng 0 biểu thị cảm biến đang mất tín hiệu, để node màn hình hiển thị trạng thái “không có dữ liệu” thay vì giữ lại giá trị cũ.
- Tần suất gửi 2 Hz (500 ms), tách biệt với chu kỳ đo/lọc cục bộ 10 Hz (100 ms): bộ lọc vẫn phản ứng nhanh cho cảnh báo, chỉ giảm tải đường truyền.

Vì hai dự án dùng hệ thống biên dịch khác nhau (PlatformIO/Arduino và ESP-IDF) và không dùng chung đường dẫn header, cấu trúc bản tin phải được khai báo **trùng khớp thủ công** ở cả hai phía; tài liệu `docs/architecture/ESPNOW_NETWORK.md` đóng vai trò nguồn thông tin dùng chung duy nhất về địa chỉ MAC, kênh và lược đồ dữ liệu.

5.4. Cảnh báo còi cục bộ

Còi buzzer được điều khiển trực tiếp trên `sensor-node` theo hai mức ngưỡng. Sau khi bỏ đường cloud, ngưỡng chỉ còn tồn tại một nơi duy nhất trong mã nguồn firmware nên không còn nguy cơ trôi lệch giữa còi vật lý và hiển thị màn hình:

- **WARNING** (20 – 50 cm): kêu 1 lần mỗi 3 giây.
- **DANGER** (<20 cm): kêu 1 lần mỗi 1 giây.



Hình 4: Thang ngưỡng cảnh báo SAFE / WARNING / DANGER của còi buzzer cục bộ trên node cảm biến.

6. Firmware Waveshare Screen

6.1. Nhận dữ liệu ESP-NOW và đánh giá hazard cục bộ

Vì không sử dụng Arduino core, phần mạng dùng trực tiếp API ESP-IDF: `esp_wifi.h`, `esp_event.h`, `esp_netif.h`, `esp_now.h` và `nvs_flash.h`. Logic nhận ESP-NOW nằm ngay trong `src/main.c`, **không tách thành component riêng** — `esp_now` vốn là API hệ thống của ESP-IDF, không cần lớp bọc.

Tác vụ mạng khởi tạo Wi-Fi ở chế độ STA “trần” (không kết nối điểm truy cập nào), cố định kênh, rồi đăng ký hàm gọi lại `esp_now_register_recv_cb()`. Với mỗi bản tin nhận được, từng vị trí cảm biến được cập nhật hoặc xóa tùy theo cờ hợp lệ, sau đó hàm đánh giá hazard **tự tính trạng thái cảnh báo tổng thể** từ sáu vị trí cảm biến — thay cho vai trò của Rule-Chain trên đám mây trước đây.

Do ESP-NOW không có cơ chế *heartbeat* ở tầng giao thức, một bộ định thời (`esp_timer`) chu kỳ 1 giây theo dõi thời điểm nhận gói cuối cùng; nếu quá 1,5 giây không nhận được bản tin nào, phù hiệu trạng thái trên giao diện chuyển sang “NO LINK” để người vận hành biết liên kết đã mất.

6.2. Mô hình dữ liệu cảm biến

Component `sensor_model` lưu trạng thái 6 cảm biến (khoảng cách, góc lắp, cờ dữ liệu cũ), được bảo vệ bằng một mutex FreeRTOS dùng chung. Mỗi khoảng cách được phân loại thành ba vùng: **SAFE** (>100 cm), **CAUTION** (30 – 100 cm), **DANGER** (<30 cm).

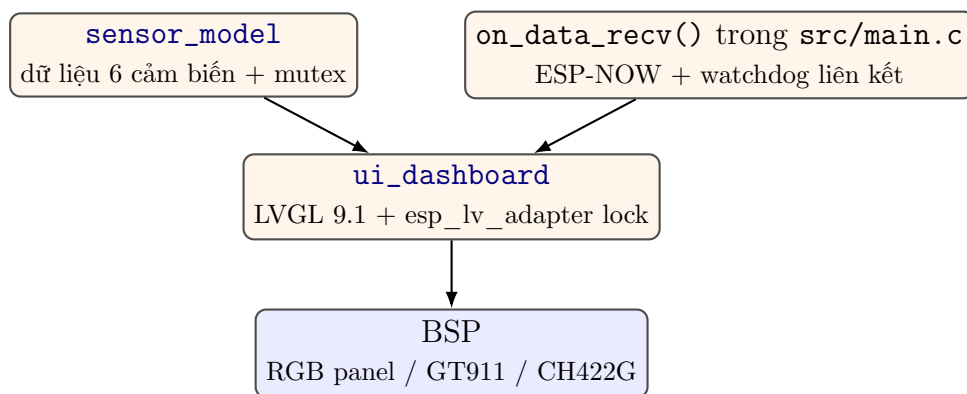
6.3. Giao diện LVGL 9.1

Giao diện sử dụng các widget LVGL: `lv_arc` (biểu diễn vùng cảm biến quanh xe), `lv_label`, `lv_anim` (hiệu ứng cảnh báo), `lv_timer` (làm mới thông tin hệ thống định kỳ) và bố cục flex. Vì các tác vụ mạng chạy trên luồng (task) khác với luồng render

LVGL, mọi thao tác cập nhật giao diện từ tác vụ mạng đều được bảo vệ bằng cơ chế khoá `esp_lv_adapter_lock()` với thời gian chờ giới hạn 100 ms, đảm bảo một luồng bị treo không làm treo toàn hệ thống.

6.4. Lớp hỗ trợ phần cứng màn hình (BSP)

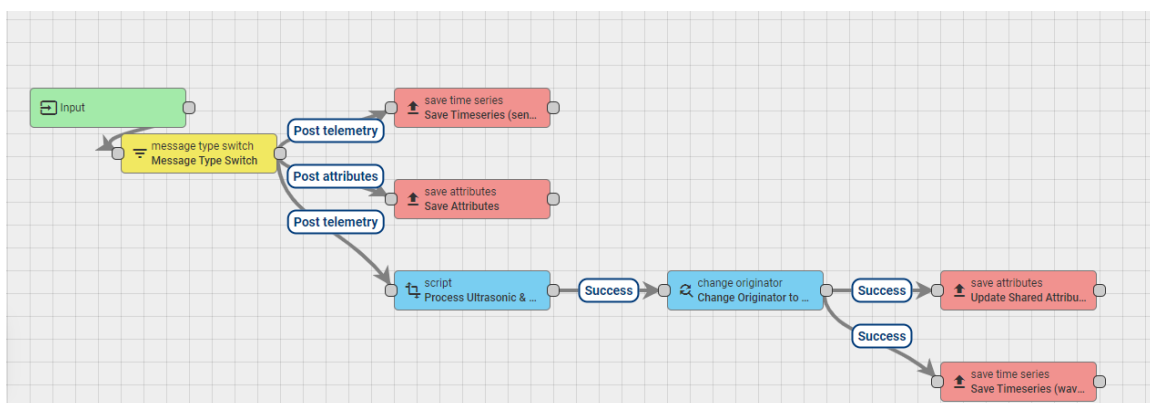
Panel RGB 800×480 (RGB565) được khởi tạo qua `esp_lcd_new_rgb_panel()`, frame-buffer đặt trong PSRAM. Bus I2C dùng API `driver/i2c_master.h`, dùng chung cho IO-expander CH422G (ghi thanh ghi trực tiếp, không có driver riêng) và cảm ứng GT911 (driver chính thức `esp_lcd_touch_gt911`) — mã nguồn: [waveshare_rgb_lcd_port.c](#).



Hình 5: Sơ đồ khối phần mềm của `waveshare-screen`.

7. Kiến trúc CoreIoT trước đây (không còn sử dụng)

Mục này mô tả kiến trúc **trước khi chuyển sang ESP-NOW**, giữ lại làm bối cảnh thiết kế và tài liệu tham chiếu cho khả năng khôi phục đường MQTT song song về sau. Rule-Chain hiện **không nằm trên đường dữ liệu**.



Hình 6: Rule-Chain CoreIoT của kiến trúc trước đây: xử lý dữ liệu telemetry và định tuyến sang node màn hình.

Rule-Chain ([supersonic_rule_chain.json](#)) nhận bản tin telemetry từ `sensor-node`, script biến đổi (JavaScript) tính khoảng cách gần nhất trong các cảm biến, suy ra trạng thái `vehicle_detected`, `warning_status` (NORMAL/WARNING/DANGER) và `relay`; sau đó

đổi chủ thể bản tin sang thiết bị **waveshare-screen** và đẩy dữ liệu xuống bằng cơ chế MQTT Shared Attributes (**notifyDevice=true**).

Lý do thay thế: ESP-NOW và Wi-Fi STA dùng chung radio, nên không thể vừa duy trì kết nối tới điểm truy cập vừa giữ liên kết ESP-NOW ổn định nếu hai bên khác kênh. Việc giữ song song hai đường truyền đòi hỏi đánh đổi độ ổn định, trong khi lợi ích của đường cloud (lưu lịch sử, giám sát từ xa) không thiết yếu với chức năng cảnh báo va chạm thời gian thực. Đổi lại, ngưỡng cảnh báo nay chỉ tồn tại trong firmware nên không còn nguy cơ trôi lệch giữa mã nguồn và cấu hình đám mây.

8. Chức năng đã hoàn thiện

- Đo và lọc khoảng cách ổn định từ **đầy đủ 6 cảm biến** siêu âm bao quanh xe.
- Truyền dữ liệu trực tiếp giữa hai board bằng ESP-NOW, hoạt động độc lập với hạ tầng mạng.
- Đánh giá ngưỡng cảnh báo **cục bộ ngay trên node màn hình**, không phụ thuộc xử lý phía đám mây.
- Phát hiện mất liên kết bằng bộ định thời giám sát, hiển thị rõ trạng thái “NO LINK” trên giao diện.
- Dashboard LVGL hiển thị trạng thái va chạm theo thời gian thực, kèm thông tin hệ thống (phiên bản firmware, bộ nhớ, thời gian hoạt động).
- Cảnh báo còi cục bộ trên **sensor-node**, phản hồi tức thời không phụ thuộc độ trễ mạng.
- Cơ chế khoá luồng an toàn giữa tác vụ mạng và tác vụ giao diện trên node màn hình.

9. Hướng phát triển tiếp theo

- **Kiểm chứng tầm hoạt động ESP-NOW khi lắp cố định trên xe**: khoảng cách giữa hai board chỉ vài mét nên tầm xa không phải giới hạn, nhưng khung và động cơ kim loại có thể gây che chắn/nhiều đáng kể — cần đo thực tế sau khi lắp đặt hoàn chỉnh.
- **Bật mã hoá cho liên kết ESP-NOW**: hiện đang truyền không mã hoá (**encrypt = false**). Chấp nhận được ở phạm vi cục bộ trong xe, nhưng nên bật CCMP (AES-128) nếu mở rộng phạm vi hoặc truyền thêm dữ liệu nhạy cảm.
- **Kênh điều khiển ngược từ màn hình về node cảm biến**: nút “Mute Alarm” hiện chỉ tắt cảnh báo hình ảnh trên màn hình, chưa tắt được còi vật lý — cần bổ sung liên kết ESP-NOW hai chiều.
- **Dọn hai dòng trạng thái tồn dư của kiến trúc MQTT trên giao diện**: dòng BUZZER và RELAY vốn hiển thị kết quả do Rule-Chain tính; sau khi bỏ đường đám mây, chúng không còn nguồn dữ liệu nên đứng yên vĩnh viễn. Cần xoá khỏi giao diện, hoặc bổ sung các trạng thái này vào bản tin ESP-NOW để hiển thị đúng.
- Bổ sung cảnh báo cho người đi đường phía ngoài xe (còi/đèn ngoài xe).